

SummarAI

Individual Reflection

Bojana

NLP Group Project · 2025–2026

Individual Reflection — Bojana

My Contributions

Backend and NLP Pipeline

I built the entire server-side application (`src/app.py`). This includes the FastAPI server, the file upload endpoint, and the full two-stage NLP pipeline: routing audio files through `faster-whisper` for transcription, then passing the transcript to Claude Haiku 4.5 to extract the summary, typed takeaways, and action items. I also built all project management endpoints — creating, listing, and deleting projects and saved summarizations — using a thin HTTP wrapper to avoid adding a full Supabase client library as a dependency.

Input type routing: Audio and video files need to be written to a temporary file on disk before Whisper can process them. Text files are read directly into memory. Temporary files are cleaned up reliably using `try/finally` even when transcription fails.

System Prompt and Output Schema

I designed the system prompt and the JSON output schema that enforces the structure Claude must return. The schema defines three required fields — `summary`, `key_takeaways`, and `action_items` — with strict types and no additional properties allowed. Early versions produced inconsistent output: action items appearing inside the summary, takeaways missing their type tag, or the model inventing owners not in the transcript. I iterated on the prompt until the model reliably separated the three output types and only included owners and deadlines when explicitly stated. The final prompt and schema are saved in `prompts/`.

Labeled Transcripts

I created approximately 8 transcripts for `eval_set.jsonl`, including standup meetings, sprint planning sessions, and one edge case where a speaker discusses a hypothetical future action without committing — designed to test whether the model invents phantom action items.

What I Learned

This project gave me a real understanding of prompt engineering as an NLP technique rather than just a trick. When we started, the model would sometimes invent owners not mentioned in the transcript, or merge two separate action items into one. Fixing this required understanding *why* the model was doing it — it was following conversational implicature, inferring responsibility from context the way a human would — and then writing a prompt constraint specific enough to override that default behavior. I also came to appreciate what makes LLM-based information extraction different from traditional approaches: the model handles paraphrase, messy grammar, and implicit references naturally. The cost is that you cannot fully predict or control the output, which is exactly why structured schemas and evaluation against gold labels are necessary.